

0.28. Aller à une case : va_a

`va_a(x, y, tuile, colonne, ligne)` : on ne compte plus les pas, on dit où aller.

1. Ce qu'il faut comprendre

Jusqu'ici, on comptait les pas. Pour aller en (10, 5) depuis (3, 2), il faudrait calculer $10 - 3 = 7$ et $5 - 2 = 3$.

`va_a` fait ce calcul pour nous.

Ce qui est nouveau ici : `va_a(colonne, ligne, tuile, versColonne, versLigne)`. Les deux derniers nombres ne sont plus des pas, mais **la case d'arrivée**.

Le chemin : à chaque quart de seconde, un pas vers la colonne voulue (si elle n'y est pas) **et** un pas vers la ligne voulue (si elle n'y est pas). De (0, 0) à (10, 5) : en **diagonale** jusqu'en (5, 5), où la ligne est bonne, puis **tout droit** jusqu'en (10, 5).

Les positions sont rangées toutes seules, comme avec `deplace` : écris-la seule sur sa ligne.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1668 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;           // la COLONNE de la lettre
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;           // la LIGNE de la lettre
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    //  va_a(x, y, ALPHABET[0], 10, 5);
    //      | | |           | |
    //      | | |           |  +--- la LIGNE d'arrivée : 5
    //      | | |           +----- la COLONNE d'arrivée : 10
    //      | | +----- tuile : la lettre A
    //      +---+----- d'où elle part ; la console y range
    //                      l'arrivée : x = 10, y = 5
    //
    //  A                      (0, 0)
    //      ↘
    //      ↘
    //      ↘
    //      ↘
    //      → → → → → A    ...puis tout droit : (5, 5) → (10, 5)
    va_a(x, y, ALPHABET[0], 10, 5);
    nombre(0, 17, x);      // 010
    nombre(4, 17, y);      // 005

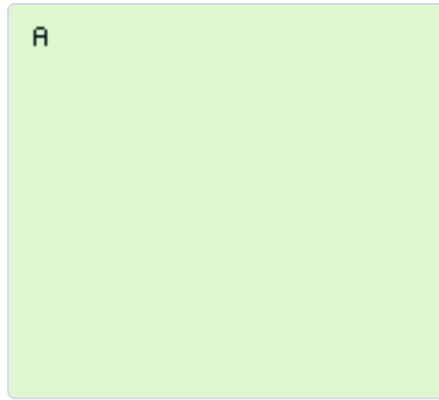
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



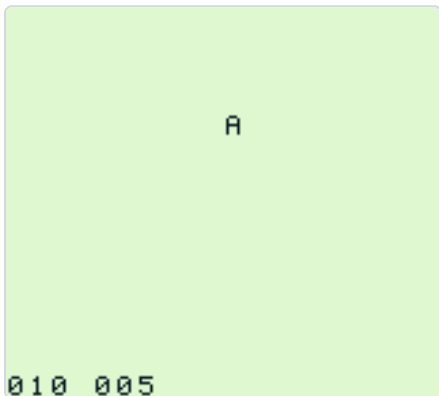
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend en biais jusqu'en (5, 5), puis file tout droit jusqu'en (10, 5).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la diagonale d'abord : (3, 3), (5, 5)
- ✓ puis tout droit : (8, 5)
- ✓ arrivé en (10, 5) ; $x = 10$, $y = 5$ (10,5)

Ensuite : **0.28.1. Aller à une case : `va_a` — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++